

ProofPay

Spécifications techniques détaillées

1. Objectif technique

L'objectif technique est de concevoir une plateforme modulaire, sécurisée et évolutive, permettant de démarrer rapidement avec un MVP web/PWA, puis d'évoluer vers des intégrations de paiement réelles, une meilleure automatisation et un déploiement multi-pays. La solution doit rester lisible pour une petite équipe de développement, tout en étant suffisamment structurée pour supporter la croissance.

2. Architecture cible

- Front-end : Angular en mode SPA ou PWA afin d'offrir une expérience mobile-first sans dépendre immédiatement d'applications natives.
- Back-end : Spring Boot avec architecture en couches ou hexagonale légère (API, service, domaine, persistance).
- Base de données : PostgreSQL pour la robustesse transactionnelle et la simplicité d'administration.
- Authentification : JWT avec OTP pour l'identification initiale.
- Stockage des pièces : stockage objet ou répertoire sécurisé selon l'environnement.
- Intégrations externes : adaptateurs de paiement, SMS, e-mail, WhatsApp, logs/monitoring.

3. Découpage applicatif

Module	Responsabilités
User / Identity	Gestion des comptes, OTP, profil, statut de vérification, suspension
Transaction	Création de transaction, calcul de frais, transitions d'état, validations métier
Payment	Initialisation de paiement, réception des callbacks, rapprochement, blocage logique e
Dispute	Ouverture de litige, ajout de preuves, workflow support, décisions d'arbitrage
Notification	Envoi de messages transactionnels vers SMS, e-mail, WhatsApp
Admin / Reporting	Recherche, supervision, administration des paramètres, journaux d'audit
Files / Evidence	Téléversement et accès sécurisé aux pièces jointes

4. Proposition d'architecture logique

- Couche API REST : expose les endpoints sécurisés consommés par le front-end et par les systèmes externes.
- Couche service métier : porte les règles métier (statuts, délais, validations, ouverture de litige, relâche automatique).
- Couche domaine : entités, énumérations, objets valeur, politiques métier.
- Couche persistance : repositories JPA ou SQL, requêtes de recherche, journalisation d'audit.
- Adaptateurs externes : implémentent les connecteurs de paiement et notification.

5. Schéma de données – entités principales

Table	Description	Champs clés
-------	-------------	-------------

users	Compte principal	id, phone, first_name, last_name, status, rating, created_at
user_verifications	OTP/KYC/états de contrôle	id, user_id, type, status, verified_at
transactions	Cœur métier	id, public_ref, buyer_id, seller_id, amount, fees, status, delivery
transaction_events	Piste d'audit	id, transaction_id, event_type, payload, created_by, created_at
payments	Paielements et rapprochements	id, transaction_id, provider, request_ref, provider_ref, status
payment_attempts	Tentatives de paiement	id, payment_id, channel, amount, result, created_at
disputes	Litiges	id, transaction_id, opened_by, reason, status, decision, closed_at
attachments	Pièces jointes	id, owner_type, owner_id, path, mime_type, uploaded_by, created_at
notifications	Messages transactionnels	id, user_id, channel, template_code, status, sent_at
admin_settings	Paramètres métier	key, value, scope, updated_at

6. Détail des champs critiques

Champ	Type suggéré	Usage / règle
transactions.public_ref	VARCHAR unique	Référence visible par les utilisateurs et le support
transactions.status	ENUM	Doit suivre la machine d'état définie
transactions.confirmation_mode	ENUM	BUTTON, OTP, CODE_SECRET
transactions.auto_release_at	TIMESTAMP	Date/heure de libération automatique si pas de contestation
payments.status	ENUM	INITIATED, PENDING, CONFIRMED, FAILED, REVERSED
disputes.status	ENUM	OPEN, UNDER_REVIEW, RESOLVED, REJECTED
users.status	ENUM	ACTIVE, SUSPENDED, BLOCKED, PENDING_VERIFICATION

7. API REST – proposition d'endpoints

Méthode	Endpoint	Description
POST	/api/auth/request-otp	Demande d'OTP pour connexion/inscription
POST	/api/auth/verify-otp	Validation OTP et émission du JWT
GET	/api/users/me	Profil du compte connecté
POST	/api/transactions	Créer une transaction
GET	/api/transactions/{id}	Détail d'une transaction
POST	/api/transactions/{id}/accept	Accepter une transaction
POST	/api/transactions/{id}/reject	Refuser une transaction
POST	/api/transactions/{id}/pay	Initialiser le paiement
POST	/api/payments/callback/{provider}	Recevoir un callback prestataire
POST	/api/transactions/{id}/mark-delivered	Déclarer la livraison/prestation
POST	/api/transactions/{id}/confirm	Confirmer la réception
POST	/api/transactions/{id}/open-dispute	Ouvrir un litige
POST	/api/disputes/{id}/decision	Décision administrative
GET	/api/admin/transactions	Recherche support

8. Principales règles techniques de transition

- Les transitions d'état doivent être centralisées dans un service métier unique pour éviter les incohérences.
- Chaque changement d'état doit générer un événement technique et un événement fonctionnel horodaté.
- Les callbacks de paiement doivent être idempotents : rejouer la même notification ne doit pas produire d'effet double.
- Les opérations sensibles (confirmation finale, décision de litige, remboursement) doivent être journalisées avec l'identité de l'acteur et l'horodatage.
- Toute action interdite par le statut courant doit renvoyer une erreur métier claire.

9. Sécurité

- JWT avec durée de vie courte, refresh token optionnel ou reconnexion OTP selon la stratégie choisie.

- Vérification OTP pour les opérations de connexion et confirmation sensible si nécessaire.
- Hashage robuste des secrets/mots de passe si un mot de passe est conservé.
- Contrôle d'accès par rôle : USER, ADMIN, SUPPORT.
- Validation systématique des entrées côté API.
- Protection contre la répétition involontaire de requêtes (idempotency key pour paiement et confirmation).
- Chiffrement des informations sensibles en stockage si requis.
- Journalisation de sécurité : tentatives OTP, connexions, suspensions, décisions de support.

10. Intégration paiement – stratégie d'implémentation

- Définir une interface PaymentProvider avec méthodes initPayment(), verifyPayment(), refund(), parseCallback().
- Créer un adaptateur par prestataire (ex. MTN, Moov, Orange) sans polluer le service métier principal.
- Commencer avec un MockPaymentProvider pour le développement et les démonstrations.
- Conserver séparément la référence interne ProofPay et la référence externe du prestataire.
- Mettre en place une tâche de rapprochement périodique pour vérifier les paiements en attente trop longtemps.

11. Notifications et templates

- Les notifications doivent être asynchrones pour ne pas bloquer les opérations métier.
- Prévoir un NotificationService piloté par file ou exécution différée légère.
- Les templates doivent être versionnés et paramétrables (nom, montant, référence, statut).
- Conserver l'historique d'envoi et la preuve d'échec pour faciliter le support.

12. Stockage des pièces justificatives

- Stockage objet recommandé (S3-compatible, serveur de fichiers ou service cloud), avec métadonnées en base.
- Contrôle des types acceptés : images, PDF, captures utiles au litige.
- Taille maximale et nettoyage de contenu dangereux à prévoir.
- Les URLs d'accès doivent être protégées, signées ou servies via une API autorisée.

13. Performances et disponibilité

- Le MVP doit privilégier la simplicité, mais les endpoints les plus utilisés doivent être paginés et indexés.
- Index recommandés : transaction public_ref, status, buyer_id, seller_id, created_at ; payment provider_ref ; dispute status.
- Les opérations longues (notification, rapprochement, libération automatique) doivent pouvoir être exécutées en tâche de fond.
- Prévoir des timeouts et mécanismes de retry pour les appels externes.

14. Observabilité et exploitation

- Logs structurés corrélés par transaction et par identifiant de requête.

- Métriques à suivre : nombre de transactions créées, taux de paiement confirmé, nombre de litiges, callbacks en erreur.
- Alertes minimales : callback indisponible, taux d'échec OTP, saturation stockage pièces, tâche de relâche automatique en erreur.
- Journal d'audit métier distinct des logs techniques.

15. Déploiement et environnements

- Environnements recommandés : local, intégration, préproduction, production.
- Conteneurisation Docker recommandée pour le backend et le front.
- Reverse proxy / API gateway selon l'infrastructure disponible.
- Variables de configuration externalisées pour les secrets et endpoints externes.

16. Stratégie de tests

- Tests unitaires pour les transitions de statut et règles métier.
- Tests d'intégration pour l'API REST et les repositories.
- Tests de contrat pour les callbacks de paiement simulés.
- Tests E2E des principaux scénarios : création, paiement, confirmation, litige.
- Jeux de données de démonstration pour les recettes fonctionnelles.

17. Risques techniques et mesures de réduction

- Risque d'incohérence de statut -> centraliser toutes les transitions dans un seul service et journaliser chaque événement.
- Risque de double traitement du callback -> idempotence obligatoire.
- Risque réglementaire lié aux flux financiers -> isoler les composants de paiement et documenter précisément la logique future d'escrow réel.
- Risque de pièces litigieuses volumineuses -> limiter la taille et prévoir une politique de conservation.